

aliasname

is the name of the alias to be created.

Usage

The symbol *name* must already be defined in the source file before creating an alias for it. Properties of *name* set by the `EXPORT` directive are not inherited by *aliasname*, so you must use `EXPORT` on *aliasname* if you want to make the alias available outside the current source file. Apart from the properties set by the `EXPORT` directive, *name* and *aliasname* are identical.

Correct example

```
baz
bar PROC
    BX lr
    ENDP
    ALIAS bar,foo      ; foo is an alias for bar
    EXPORT bar
    EXPORT foo         ; foo and bar have identical properties
                        ; because foo was created using ALIAS
    EXPORT baz         ; baz and bar are not identical
                        ; because the size field of baz is not set
```

Incorrect example

```
EXPORT bar
IMPORT car
ALIAS bar,foo ; ERROR - bar is not defined yet
ALIAS car,boo ; ERROR - car is external
bar PROC
    BX lr
    ENDP
```

Related information

[EXPORT](#) or [GLOBAL](#) on page 1448

22.5 ALIGN

The `ALIGN` directive aligns the current location to a specified boundary by padding with zeros or NOP instructions.

Syntax

```
ALIGN {expr {, {<offset{,pad>}{,padsiz>}}}}
```

where:

expr

is a numeric expression evaluating to any power of 2 from 20 to 231

offset

can be any numeric expression

pad

can be any numeric expression

padsiz

can be 1, 2 or 4.

Operation

The current location is aligned to the next lowest address of the form:

$$\text{offset} + n * \text{expr}$$

n is any integer which the assembler selects to minimise padding.

If *expr* is not specified, `ALIGN` sets the current location to the next word (four byte) boundary. The unused space between the previous and the new current location are filled with:

- Copies of *pad*, if *pad* is specified.
- `NOB` instructions, if all the following conditions are satisfied:
 - *pad* is not specified.
 - The `ALIGN` directive follows A32 or T32 instructions.
 - The current section has the `CODEALIGN` attribute set on the `AREA` directive.
- Zeros otherwise.

pad is treated as a byte, halfword, or word, according to the value of *padsiz*. If *padsiz* is not specified, *pad* defaults to bytes in data sections, halfwords in T32 code, or words in A32 code.

Usage

Use `ALIGN` to ensure that your data and code is aligned to appropriate boundaries. This is typically required in the following circumstances:

- The `ADR` T32 pseudo-instruction can only load addresses that are word aligned, but a label within T32 code might not be word aligned. Use `ALIGN 4` to ensure four-byte alignment of an address within T32 code.
- Use `ALIGN` to take advantage of caches on some Arm processors. For example, the Arm940T has a cache with 16-byte lines. Use `ALIGN 16` to align function entries on 16-byte boundaries and maximize the efficiency of the cache.
- A label on a line by itself can be arbitrarily aligned. Following A32 code is word-aligned (T32 code is halfword aligned). The label therefore does not address the code correctly. Use `ALIGN 4` (or `ALIGN 2` for T32) before the label.

Alignment is relative to the start of the ELF section where the routine is located. The section must be aligned to the same, or coarser, boundaries. The `ALIGN` attribute on the `AREA` directive is specified differently.

Examples

```

        AREA    cacheable, CODE, ALIGN=3
rout1   ; code          ; aligned on 8-byte boundary

```

```

; code
MOV    pc,lr    ; aligned only on 4-byte boundary
ALIGN  8        ; now aligned on 8-byte boundary
rout2  ; code

```

In the following example, the `ALIGN` directive tells the assembler that the next instruction is word aligned and offset by 3 bytes. The 3 byte offset is counted from the previous word aligned address, resulting in the second `DCB` placed in the last byte of the same word and 2 bytes of padding are to be added.

```

AREA    OffsetExample, CODE
DCB     1      ; This example places the two bytes in the first
ALIGN   4,3    ; and fourth bytes of the same word.
DCB     1      ; The second DCB is offset by 3 bytes from the
               ; first DCB.

```

In the following example, the `ALIGN` directive tells the assembler that the next instruction is word aligned and offset by 2 bytes. Here, the 2 byte offset is counted from the next word aligned address, so the value `n` is set to 1 (`n=0` clashes with the third `DCB`). This time three bytes of padding are to be added.

```

AREA    OffsetExample1, CODE
DCB     1      ; In this example, n cannot be 0 because it
DCB     1      ; clashes with the 3rd DCB. The assembler
DCB     1      ; sets n to 1.
ALIGN   4,2    ; The next instruction is word aligned and
DCB     2      ; offset by 2.

```

In the following example, the `DCB` directive makes the PC misaligned. The `ALIGN` directive ensures that the label `subroutine1` and the following instruction are word aligned.

```

start   AREA    Example, CODE, READONLY
        LDR     r6,=label1
        ; code
        MOV     pc,lr
label1  DCB     1      ; PC now misaligned
        ALIGN   ; ensures that subroutine1 addresses
subroutine1 ; the following instruction.
        MOV     r5,#0x5

```

Related information

[AREA](#) on page 1428

22.6 AREA

The `AREA` directive instructs the assembler to assemble a new code or data section.

Syntax

```
AREA sectionname ,{attr},{attr}...
```

where:

sectionname

is the name to give to the section. Sections are independent, named, indivisible chunks of code or data that are manipulated by the linker.

You can choose any name for your sections. However, names starting with a non-alphabetic character must be enclosed in bars or a missing section name error is generated. For example, `|1_DataArea|`.

Certain names are conventional. For example, `|.text|` is used for code sections produced by the C compiler, or for code sections otherwise associated with the C library.

attr

are one or more comma-delimited section attributes. Valid attributes are:

ALIGN=expression

By default, ELF sections are aligned on a four-byte boundary. *expression* can have any integer value from 0 to 31. The section is aligned on a $2^{\text{expression}}$ -byte boundary. For example, if *expression* is 10, the section is aligned on a 1KB boundary.

This is not the same as the way that the `ALIGN` directive is specified.



Do not use `ALIGN=0` or `ALIGN=1` for A32 code sections.

Do not use `ALIGN=0` for T32 code sections.

ASSOC=section

section specifies an associated ELF section. *sectionname* must be included in any link that includes *section*.

CODE

Contains machine instructions. `READONLY` is the default.

CODEALIGN

Causes `armasm` to insert `NOP` instructions when the `ALIGN` directive is used after A32 or T32 instructions within the section, unless the `ALIGN` directive specifies a different padding. `CODEALIGN` is the default for execute-only sections.

COMDEF

Is a common section definition. This ELF section can contain code or data. It must be identical to any other section of the same name in other source files.

Identical ELF sections with the same name are overlaid in the same section of memory by the linker. If any are different, the linker generates a warning and does not overlay the sections.

COMGROUP=symbol_name

Is the signature that makes the `AREA` part of the named ELF section group. See the `GROUP=symbol_name` for more information. The `COMGROUP` attribute marks the ELF section group with the `GRP_COMDAT` flag.

COMMON

Is a common data section. You must not define any code or data in it. It is initialized to zeros by the linker. All common sections with the same name are overlaid in the same section of memory by the linker. They do not all have to be the same size. The linker allocates as much space as is required by the largest common section of each name.

DATA

Contains data, not instructions. `READWRITE` is the default.

EXECONLY

Indicates that the section is execute-only. Execute-only sections must also have the `CODE` attribute, and must not have any of the following attributes:

- `READONLY`.
- `READWRITE`.
- `DATA`.
- `ZEROALIGN`.

`armasm` faults if any of the following occur in an execute-only section:

- Explicit data definitions, for example `DCD` and `DCB`.
- Implicit data definitions, for example `LDR r0, =0xaabbccdd`.
- Literal pool directives, for example `LTORG`, if there is literal data to be emitted.
- `INCBIN` or `SPACE` directives.
- `ALIGN` directives, if the required alignment cannot be accomplished by padding with `NOP` instructions. `armasm` implicitly applies the `CODEALIGN` attribute to sections with the `EXECONLY` attribute.

FINI_ARRAY

Sets the ELF type of the current area to `SHT_FINI_ARRAY`.

GROUP=symbol_name

Is the signature that makes the `AREA` part of the named ELF section group. It must be defined by the source file, or a file included by the source file. All `AREAS` with the same `symbol_name` signature are part of the same group. Sections within a group are kept or discarded together.

INIT_ARRAY

Sets the ELF type of the current area to `SHT_INIT_ARRAY`.

LINKORDER=section

Specifies a relative location for the current section in the image. It ensures that the order of all the sections with the `LINKORDER` attribute, with respect to each other, is the same as the order of the corresponding named *sections* in the image.

MERGE=n

Indicates that the linker can merge the current section with other sections with the `MERGE=:samp:{n}` attribute. *n* is the size of the elements in the section, for example

n is 1 for characters. You must not assume that the section is merged, because the attribute does not force the linker to merge the sections.

NOALLOC

Indicates that no memory on the target system is allocated to this area.

NOINIT

Indicates that the data section is uninitialized, or initialized to zero. It contains only space reservation directives `SPACE` or `DCB`, `DCD`, `DCDU`, `DCQ`, `DCQU`, `DCW`, or `DCWU` with initialized values of zero. You can decide at link time whether an area is uninitialized or zero-initialized.



Arm® Compiler does not support systems with ECC or parity protection where the memory is not initialized.

PREINIT_ARRAY

Sets the ELF type of the current area to `SHT_PREINIT_ARRAY`.

READONLY

Indicates that this section must not be written to. This is the default for Code areas.

READWRITE

Indicates that this section can be read from and written to. This is the default for Data areas.

SECFLAGS= n

Adds one or more ELF flags, denoted by n , to the current section.

SECTYPE= n

Sets the ELF type of the current section to n .

STRINGS

Adds the `SHT_STRINGS` flag to the current section. To use the `STRINGS` attribute, you must also use the `MERGE=1` attribute. The contents of the section must be strings that are nul-terminated using the `DCB` directive.

ZEROALIGN

Causes `armasm` to insert zeros when the `ALIGN` directive is used after A32 or T32 instructions within the section, unless the `ALIGN` directive specifies a different padding. `ZEROALIGN` is the default for sections that are not execute-only.

Usage

Use the `AREA` directive to subdivide your source file into ELF sections. You can use the same name in more than one `AREA` directive. All areas with the same name are placed in the same ELF section. Only the attributes of the first `AREA` directive of a particular name are applied.

In general, Arm recommends that you use separate ELF sections for code and data. However, you can put data in code sections. Large programs can usually be conveniently divided into several code sections. Large independent data sets are also usually best placed in separate sections.

The scope of numeric local labels is defined by `AREA` directives, optionally subdivided by `ROUT` directives.

There must be at least one `AREA` directive for an assembly.



armasm emits `R_ARM_TARGET1` relocations for the `DCD` and `DCDU` directives if the directive uses PC-relative expressions and is in any of the `PREINIT_ARRAY`, `FINI_ARRAY`, or `INIT_ARRAY` ELF sections. You can override the relocation using the `RELOC` directive after each `DCD` or `DCDU` directive. If this relocation is used, read-write sections might become read-only sections at link time if the platform ABI permits this.

Example

The following example defines a read-only code section named `Example`:

```
AREA    Example, CODE, READONLY    ; An example code section.
; code
```

Related information

[ALIGN](#) on page 1426

[RELOC](#) on page 1478

[DCD and DCDU](#) on page 1439

[ELF sections and the AREA directive](#) on page 99

[Information about image structure and generation](#)

22.7 ARM or CODE32 directive

The `ARM` directive instructs the assembler to interpret subsequent instructions as A32 instructions, using either the UAL or the pre-UAL Arm assembler language syntax. `CODE32` is a synonym for `ARM`.



Not supported for AArch64 state.

Syntax

`ARM`

Usage

In files that contain code using different instruction sets, the `ARM` directive must precede any A32 code.

22.22 DCW and DCWU

The `DCW` directive allocates one or more halfwords of memory, aligned on two-byte boundaries, and defines the initial runtime contents of the memory. `DCWU` is the same, except that the memory alignment is arbitrary.

Syntax

```
{label} DCWU expr, {expr}...
```

where:

expr

is a numeric expression that evaluates to an integer in the range -32768 to 65535.

Usage

`DCW` inserts a byte of padding before the first defined halfword if necessary to achieve two-byte alignment.

Use `DCWU` if you do not require alignment.

Examples

```
data    DCW    -225,2*number    ; number must already be defined
        DCWU   number+4
```

Related information

[DCB](#) on page 1438

[DCD and DCDU](#) on page 1439

[DCQ and DCQU](#) on page 1443

[SPACE or FILL](#) on page 1485

[Numeric expressions](#) on page 228

22.23 END

The `END` directive informs the assembler that it has reached the end of a source file.

Syntax

```
END
```

Usage

Every assembly language source file must end with `END` on a line by itself.

If the source file has been included in a parent file by a `GET` directive, the assembler returns to the parent file and continues assembly at the first line following the `GET` directive.

If `END` is reached in the top-level source file during the first pass without any errors, the second pass begins.

If `END` is reached in the top-level source file during the second pass, the assembler finishes the assembly and writes the appropriate output.

Related information

[GET](#) or [INCLUDE](#) on page 1462

22.24 ENDFUNC or ENDP

The `ENDFUNC` directive marks the end of an AAPCS-conforming function. `ENDP` is a synonym for `ENDFUNC`.

Related information

[FUNCTION](#) or [PROC](#) on page 1459

22.25 ENTRY

The `ENTRY` directive declares an entry point to a program.

Syntax

```
ENTRY
```

Usage

A program must have an entry point. You can specify an entry point in the following ways:

- Using the `ENTRY` directive in assembly language source code.
- Providing a `main()` function in C or C++ source code.
- Using the `armlink --entry` command-line option.

You can declare more than one entry point in a program, although a source file cannot contain more than one `ENTRY` directive. For example, a program could contain multiple assembly language source files, each with an `ENTRY` directive. Or it could contain a C or C++ file with a `main()` function and one or more assembly source files with an `ENTRY` directive.

If the program contains multiple entry points, then you must select one of them. You do this by exporting the symbol for the `ENTRY` directive that you want to use as the entry point, then using the `armlink --entry` option to select the exported symbol.

Example

```
AREA   ARMex, CODE, READONLY
ENTRY   ; Entry point for the application.
EXPORT ep1 ; Export the symbol so the linker can find it
ep1     ; in the object file.
```

```
; code  
END
```

When you invoke `armlink`, if other entry points are declared in the program, then you must specify `--entry=ep1`, to select `ep1`.

Related information

[Image entry points](#)

`--entry=location`

22.26 EQU

The `EQU` directive gives a symbolic name to a numeric constant, a register-relative value or a PC-relative value.

Syntax

```
name EQU expr, {type}
```

where:

name

is the symbolic name to assign to the value.

expr

is a register-relative address, a PC-relative address, an absolute address, or a 32-bit integer constant.

type

is optional. *type* can be any one of:

- ARM.
- THUMB.
- CODE32.
- CODE16.
- DATA.

You can use only if *type* is an absolute address. If *name* is exported, the *name* entry in the symbol table in the object file is marked as `ARM`, `THUMB`, `CODE32`, `CODE16`, or `DATA`, according to *type*. This can be used by the linker.

Usage

Use `EQU` to define constants. This is similar to the use of `#define` to define a constant in C.

`*` is a synonym for `EQU`.

Examples

```
abc EQU 2           ; Assigns the value 2 to the symbol abc.
xyz EQU label+8     ; Assigns the address (label+8) to the
                    ; symbol xyz.
fiq EQU 0x1C, CODE32 ; Assigns the absolute address 0x1C to
                    ; the symbol fiq, and marks it as code.
```

Related information

[KEEP](#) on page 1468

[EXPORT](#) or [GLOBAL](#) on page 1448

22.27 EXPORT or GLOBAL

The `EXPORT` directive declares a symbol that can be used by the linker to resolve symbol references in separate object and library files. `GLOBAL` is a synonym for `EXPORT`.

Syntax

```
EXPORT {[WEAK] }
```

```
EXPORT symbol [SIZE={n}]
```

```
EXPORT symbol [{type ,{set}]}
```

```
EXPORT symbol [attr, {type, {set}}, SIZE={n}]
```

```
EXPORT symbol [WEAK , {attr}, {type, {set}}, SIZE={n}]
```

where:

symbol

is the symbol name to export. The symbol name is case-sensitive. If *symbol* is omitted, all symbols are exported.

WEAK

symbol is only imported into other sources if no other source exports an alternative *symbol*. If `[WEAK]` is used without *symbol*, all exported symbols are weak.

attr

can be any one of:

DYNAMIC

sets the ELF symbol visibility to `STV_DEFAULT`.

PROTECTED

sets the ELF symbol visibility to `STV_PROTECTED`.

HIDDEN

sets the ELF symbol visibility to `STV_HIDDEN`.

INTERNAL
sets the ELF symbol visibility to `STV_INTERNAL`.

type
specifies the symbol type:

DATA
symbol is treated as data when the source is assembled and linked.

CODE
symbol is treated as code when the source is assembled and linked.

ELFTYPE=*n*
symbol is treated as a particular ELF symbol, as specified by the value of *n*, where *n* can be any number from 0 to 15.

If unspecified, the assembler determines the most appropriate type. Usually the assembler determines the correct type so you are not required to specify it.

set
specifies the instruction set:

ARM
symbol is treated as an A32 symbol.

THUMB
symbol is treated as a T32 symbol.

If unspecified, the assembler determines the most appropriate set.

n
specifies the size and can be any 32-bit value. If the `SIZE` attribute is not specified, the assembler calculates the size:

- For `PROC` and `FUNCTION` symbols, the size is set to the size of the code until its `ENDP` or `ENDFUNC`.
- For other symbols, the size is the size of instruction or data on the same source line. If there is no instruction or data, the size is zero.

Usage

Use `EXPORT` to give code in other files access to symbols in the current file.

Use the `[WEAK]` attribute to inform the linker that a different instance of *symbol* takes precedence over this one, if a different one is available from another source. You can use the `[WEAK]` attribute with any of the symbol visibility attributes.

Examples

```
AREA Example, CODE, READONLY
EXPORT DoAdd                ; Export the function name
                             ; to be used by external modules.
DoAdd ADD r0, r0, r1
```

Related information

[Relational operators](#) on page 236

[About assembly control directives](#) on page 1423

22.45 IMPORT and EXTERN

The `IMPORT` and `EXTERN` directives provide the assembler with a name that is not defined in the current assembly.

Syntax

```
{directive symbol} {[SIZE=<n>}]
```

```
{directive symbol} {[type]}
```

```
{directive symbol} [attr{,type}{,SIZE=n}]
```

```
directive symbol [WEAK {,attr }{,type}{,SIZE=n}]
```

where:

directive

can be either:

IMPORT

imports the symbol unconditionally.

EXTERN

imports the symbol only if it is referred to in the current assembly.

symbol

is a symbol name defined in a separately assembled source file, object file, or library. The symbol name is case-sensitive.

WEAK

prevents the linker generating an error message if the symbol is not defined elsewhere. It also prevents the linker searching libraries that are not already included.

attr

can be any one of:

DYNAMIC

sets the ELF symbol visibility to `STV_DEFAULT`.

PROTECTED

sets the ELF symbol visibility to `STV_PROTECTED`.

HIDDEN

sets the ELF symbol visibility to `STV_HIDDEN`.

INTERNAL

sets the ELF symbol visibility to `STV_INTERNAL`.

type

specifies the symbol type:

DATA

symbol is treated as data when the source is assembled and linked.

CODE

symbol is treated as code when the source is assembled and linked.

ELFTYPE=*n*

symbol is treated as a particular ELF symbol, as specified by the value of *n*, where *n* can be any number from 0 to 15.

If unspecified, the linker determines the most appropriate type.

n

specifies the size and can be any 32-bit value. If the `SIZE` attribute is not specified, the assembler calculates the size:

- For `PROC` and `FUNCTION` symbols, the size is set to the size of the code until its `ENDP` or `ENDFUNC`.
- For other symbols, the size is the size of instruction or data on the same source line. If there is no instruction or data, the size is zero.

Usage

The name is resolved at link time to a symbol defined in a separate object file. The symbol is treated as a program address. If `[WEAK]` is not specified, the linker generates an error if no corresponding symbol is found at link time.

If `[WEAK]` is specified and no corresponding symbol is found at link time:

- If the reference is the destination of a `B` or `BL` instruction, the value of the symbol is taken as the address of the following instruction. This makes the `B` or `BL` instruction effectively a `NOP`.
- Otherwise, the value of the symbol is taken as zero.

Example

The example tests to see if the C++ library has been linked, and branches conditionally on the result.

```
AREA    Example, CODE, READONLY
EXTERN  __CPP_INITIALIZE[WEAK] ; If C++ library linked, gets the
                                ; address of __CPP_INITIALIZE
                                ; function.
LDR     r0,=__CPP_INITIALIZE    ; If not linked, address is zeroed.
CMP     r0,#0                  ; Test if zero.
BEQ     nocplusplus            ; Branch on the result.
```

The following examples show the use of the `SIZE` attribute:

```
EXTERN symA [SIZE=4]
EXTERN symA [DATA, SIZE=4]
```

Related information

[EXPORT](#) or [GLOBAL](#) on page 1448

[ELF for the Arm Architecture](#)

22.46 INCBIN

The `INCBIN` directive includes a file within the file being assembled. The file is included as it is, without being assembled.

Syntax

```
INCBIN filename
```

where:

filename

is the name of the file to be included in the assembly. The assembler accepts pathnames in either UNIX or MS-DOS format.

Usage

You can use `INCBIN` to include data, such as executable files, literals, or any arbitrary data. The contents of the file are added to the current ELF section, byte for byte, without being interpreted in any way. Assembly continues at the line following the `INCBIN` directive.

By default, the assembler searches the current place for included files. The current place is the directory where the calling file is located. Use the `-i` assembler command-line option to add directories to the search path. File names and directory names containing spaces must not be enclosed in double quotes (" ").

Example

```
AREA    Example, CODE, READONLY
INCBIN  file1.dat           ; Includes file1 if it exists in the current
place
INCBIN  c:\project\file2.txt ; Includes file2.
```

[DCD and DCDU](#) on page 1439[DCQ and DCQU](#) on page 1443[DCW and DCWU](#) on page 1444[Numeric expressions](#) on page 228

22.65 THUMB directive

The `THUMB` directive instructs the assembler to interpret subsequent instructions as T32 instructions, using the UAL syntax.



Not supported for AArch64 state.

Syntax

`THUMB`

Usage

In files that contain code using different instruction sets, the `THUMB` directive must precede T32 code written in UAL syntax.

If necessary, this directive also inserts one byte of padding to align to the next halfword boundary.

This directive does not assemble to any instructions. It also does not change the state. It only instructs `armasm` to assemble T32 instructions as appropriate, and inserts padding if necessary.

Example

This example shows how you can use `ARM` and `THUMB` directives to switch state and assemble both A32 and T32 instructions in a single area.

```

        AREA ToT32, CODE, READONLY          ; Name this block of code
        ENTRY                                ; Mark first instruction to execute
        ARM                                  ; Subsequent instructions are A32

start
        ADR    r0, into_t32 + 1              ; Processor starts in A32 state
        BX     r0                            ; Inline switch to T32 state
        THUMB                                  ; Subsequent instructions are T32
into_t32
        MOVS   r0, #10                       ; New-style T32 instructions

```

Related information

[ARM or CODE32 directive](#) on page 1432[CODE16 directive](#) on page 1436